

Student Details

Name:	Mithielesh N
Roll Number:	DS23F3004299
Batch:	2023
Degree Level:	Diploma
Course:	Modern Application Development - 2

Project Details

Problem Statement

A multi-user academic assessment platform (Quizzo) designed to streamline exam preparation and administration. The application enables an administrator to construct a deep hierarchy of academic content (Subjects → Chapters → Quizzes → Questions) while providing students with a seamless, reactive interface to attempt timed assessments, visualize their performance analytics, and receive automated asynchronous reporting.

Approach

Step 1: Analyzed the core requirements to establish a decoupled architecture. Opted to build a strict RESTful JSON API using Flask for the backend, paired with a modern Single Page Application (SPA) using Vue.js to handle complex dynamic states like live quiz timers without page reloads.

Step 2: Designed a highly normalized relational database schema. The models include User (with Role-Based Access Control), Subject, Chapter, Quiz, and Question to maintain the academic hierarchy. The Score table acts as the associative entity mapping user attempts to quizzes, and the Notification table handles system-wide and targeted alerts.

Step 3: Structured the Flask backend into modular controllers (auth.py, admin.py, user.py) to securely route traffic. Implemented robust validation to prevent database integrity errors and ensure secure session management.

Step 4: Developed the frontend interface. Utilized Vue.js reactive variables to implement instant search filtering for the Admin directory, dynamic modal-based CRUD operations, and an auto-submitting quiz engine.

Step 5: Integrated background job processing to handle heavy lifting without blocking the UI. Configured Celery and Redis to execute user-triggered CSV data exports asynchronously.

Step 6: Implemented scheduled tasks using Celery Beat. Created cron-like jobs to automatically scan the database to send daily in-app pendency reminders to students, as well as generate comprehensive HTML performance reports on the first day of every month.

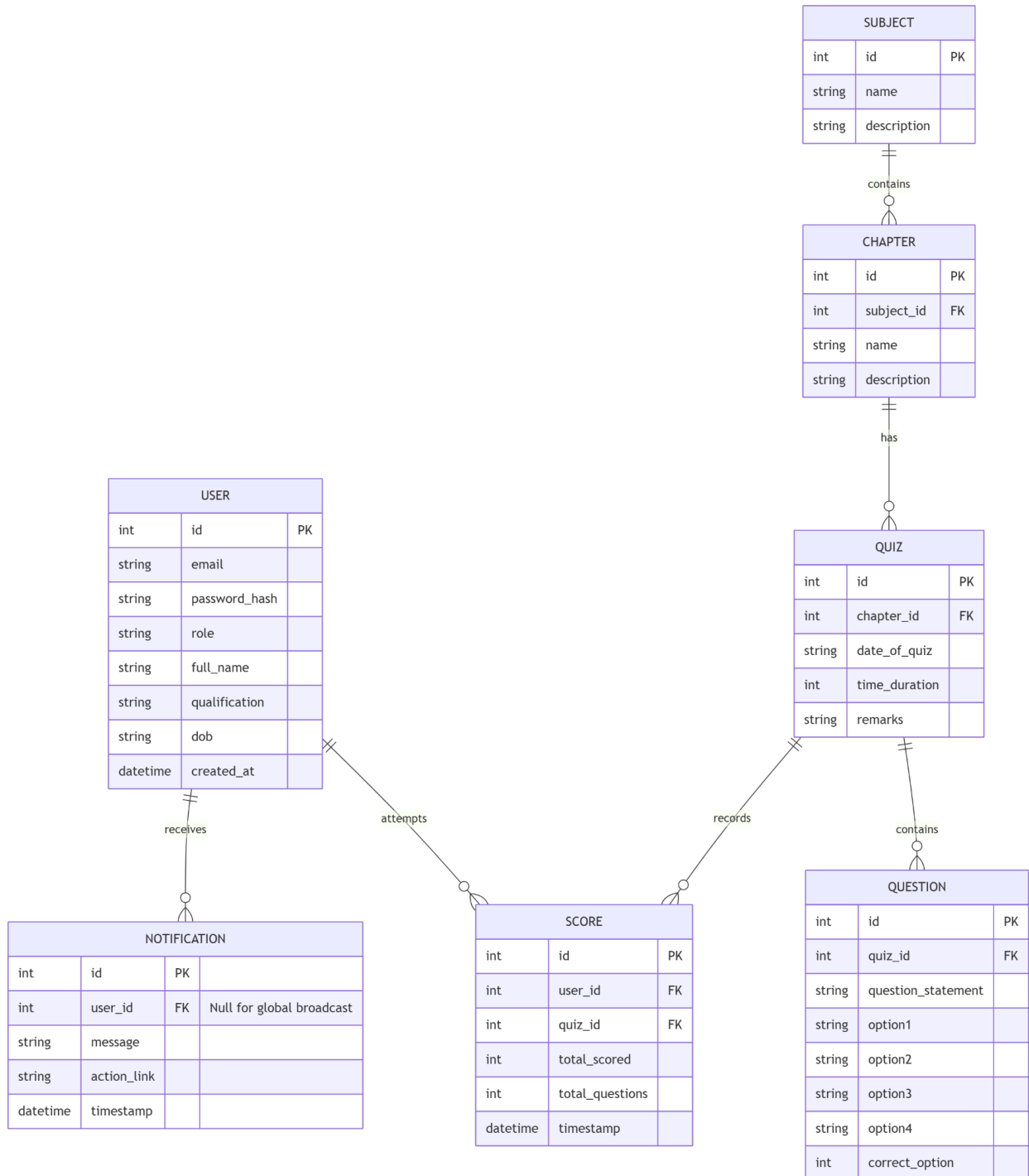
Step 7: Optimized backend performance by integrating Flask-Caching with Redis. High-traffic endpoints (such as fetching a student's available quizzes) are cached in-memory to drastically reduce SQLite database queries.

Step 8: Engineered a comprehensive Analytics Dashboard. Integrated Chart.js to consume JSON API data and render real-time visual insights, including score distribution bar charts for Admins and historical performance trend lines for Students.

Frameworks and Libraries used

1. **SQLite:** The lightweight, file-based relational database engine used for persisting all user profiles, academic hierarchies, and historical score data.
2. **Flask:** The core Python micro-framework utilized to construct the RESTful backend API and serve the initial SPA template.
3. **Vue.js (via CDN):** The progressive JavaScript framework used to build the reactive Single Page Application (SPA), managing complex client-side state and DOM updates seamlessly.
4. **Celery:** A distributed asynchronous task queue used to offload long-running processes (like CSV exports) and execute periodic scheduled jobs (monthly reports).
5. **Redis:** Serves a dual architectural purpose as the high-speed, in-memory message broker for Celery workers and the backend storage engine for Flask API caching.
6. **Bootstrap 5:** The responsive CSS framework utilized to design a modern, mobile-friendly UI with pre-built components like modals, grids, and badges.
7. **Flask-SQLAlchemy:** An extension that bridges Python objects to SQLite tables via an Object-Relational Mapper (ORM), ensuring secure, sanitized, and efficient database queries.
8. **Flask-Login:** Provides secure user session management, handling authentication state and enforcing Role-Based Access Control (RBAC) across protected API routes.
9. **Flask-Caching:** An optimization extension that dramatically reduces database load by serving frequently requested, relatively static data directly from Redis memory.
10. **Chart.js:** A client-side JavaScript charting library integrated to render dynamic, visually rich analytics dashboards directly in the browser.
11. **Werkzeug Security:** Utilized under the hood for the cryptographic hashing (Bcrypt) and verification of user passwords to ensure strict data security.

ER Diagram



Presentation Video (Drive Link)

https://drive.google.com/file/d/1S9Ewbx_cyvV94g2LSRF1yQet9K0xHqYi/view